NetEventCause: Event-Driven Root Cause Analysis for Large Network System Without Topology

Zhaolin Yuan, Long Ma, Wenjia Wei, Xia Zhu, Mingjie Sun, Duxin Chen, and Xiaojuan Ban

***Abstract*—Root cause analysis (RCA) is a crucial technique in network systems for uncovering the abnormal nodes that lead to the network alarm flood. Within private cloud network systems, the calling chains and topologies among entities, such as hosts, routes, and services, are always incomplete due to nonstandardized management. Existing topology-free RCA techniques, which rely on the casual discovery, are inapplicable when the scale of the network system is extremely large or the number of triggered alarms is sparse. This article proposes NetEventCause (NEC), an event-driven, unsupervised, and nonintrusive RCA algorithm for large network systems, where the network topology is unknown. NEC learns from historical alarm events to model the occurrences of various alarm types using a multivariate neural temporal point process (TPP). Based on the conditional intensity predicted by the learned TPP, NEC can identify the root alarms from a cascade of alarm events and locate the causal alarms of derivative alarms using the attribution method. The experimental section evaluates the NEC using both a synthetic event dataset and a large real-world dataset. The real-world dataset is exported from the Huawei Shennong Intelligent Maintenance and Operation Center (IMOC), a platform deployed at one of China's largest airports and manages over 200000 entities. Results obtained from the two datasets demonstrate that NEC outperforms most state of the art (SOTA) TPP models in modeling alarm events and surpasses general RCA methods in terms of identifying root alarms and recovering transmission chains of anomalies.**

***Index Terms*—Artificial intelligence for IT operations (AIOps), attribution method, data stream mining, event modeling.**

Received 3 May 2024; revised 31 October 2024 and 3 April 2025; accepted 19 May 2025. Date of publication 5 June 2025; date of current version 9 October 2025. This work was supported in part by the National Science and Technology Major Project of the Ministry of Science and Technology of China under Grant 2024ZD0608100; in part by the National Natural Science Foundation of China under Grant 62376025, Grant 62306032, Grant 62332017, and Grant 62273090; in part by the Zhishan Youth Scholar Program; and in part by Jiangsu Provincial Scientific Research Center of Applied Mathematics under Grant BK20233002. *(Corresponding author: Duxin Chen.)*

Zhaolin Yuan is with the Institute of Artificial Intelligence, Shunde Innovation School, University of Science and Technology Beijing, Beijing 100083, China (e-mail: yuanzhaolin@ustb.edu.cn).

Long Ma, Wenjia Wei, and Xia Zhu are with Huawei Technologies Company Ltd., Shenzhen 518129, China (e-mail: malong43@huawei.com; weiwenjia@huawei.com; zhuxia1@huawei.com).

Mingjie Sun is with the School of Computer Science and Technology, Soochow University, Suzhou 215006, China.

Duxin Chen is with the School of Mathematics, Southeast University, Nanjing 210096, China (e-mail: chendx@seu.edu.cn).

Xiaojuan Ban is with Beijing Advanced Innovation Center for Materials Genome Engineering, the School of Intelligence Science and Technology, and the Key Laboratory of Intelligent Bionic Unmanned Systems, Ministry of Education, University of Science and Technology Beijing, Beijing 100083, China, and also with the Institute of Materials Intelligent Technology, Liaoning Academy of Materials, Shenyang 110004, China (e-mail: banxj@ustb.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3574316

NOMENCLATURE

Symbol	Definition
$\mathcal{M}$	Candidate set of alarm event types.
s	Streaming alarm sequence.
$\mathbf{e}_i \in s$	Alarm event, denoted as $\mathbf{e}_i = (k_i, t_i)$.
$k_i \in \mathcal{M}$	Type of the event $\mathbf{e}_i$.
t_i	Occurring time of the event $\mathbf{e}_i$.
$\mathbf{v}_k$	Feature embedding of the event type k.
$\mathcal{H}(t)$	Historical events occurring before t.
$\lambda_k(t)$	Intensity of event type k at time t.
$\mathcal{N}_k$	Count of events with type k occurring as root events in the training dataset.
T	Total duration of the training dataset.
$\mathbf{A}_i$	Contribution vector of event $\mathbf{e}_i$.
$\hat{\mathbf{R}}^{\mathbf{e}_i}$	Inferred causative alarm set of event $\mathbf{e}_i$.

I. INTRODUCTION

IN CLOUD network systems, failures or anomalies are frequently raised by network entities, such as devices, hosts, and services. These issues are typically represented as frequent alarm events. Due to the strong relations between linked network entities, a root alarm usually causes massive derivative alarms, resulting in an alarm flood for the network operators. Root cause analysis (RCA) is a crucial technique in the field of artificial intelligence for IT operations (AIOps) [1], [2]. It aims to determine whether the reported alarm events are caused by the entities themselves or by other failures. A common challenge of RCA within private cloud systems is the incomplete calling chains and topologies due to nonstandardized management [3]. This limitation renders most existing tracing-based RCA techniques inapplicable in these scenarios [4].

To recover the missed relations between entities, previous studies introduced causal discovery algorithms to recover the missed topology. Examples of such algorithms include Peter Clark (PC) [5] and Granger causality [6] in time series, as well as event casual discovery [7], [8]. However, existing methods suffer from several challenges in discovering the causality in large network systems. First, a large private network system may contain tens of thousands of entities and the dimensions of time series and event type are too high to be afforded by most causal discovery algorithms. Meanwhile, the number of triggered alarms on the abnormal entities is sparse, so the inadequate log datasets degrade the accuracy of the restored topology. Last but not least, most event-based models

2162-237X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

employed in RCA, such as Hawkes process [9], proximal graphical event model [7], and semiparametric neural point process (SPNPP) [10], have limitations due to their strong assumptions in the change of event intensities, such as the memoryless property and decaying excitation. Although these assumptions can simplify the hypothesis space by incorporating prior constraints, these assumptions do not hold for the events in complicated network systems. The time difference between a root and its derivative alarm is influenced by various factors, such as the self-check period and network delay. This leads to complex variations in the conditional intensity function over time, rather than a simple form like the Hawkes process.

To tackle the challenges above, this article proposes a novel RCA technique, named NetEventCause (NEC), for network systems of arbitrary scale and unknown network topology. As an unsupervised RCA technique driven by log events, NEC is capable of: 1) modeling the Granger causality among alarm events in the continuous-time domain; 2) distinguishing between root alarms and derivative alarms from a cascade of alarm events; and 3) inferring the direct causative alarms of derivative ones, expressed as a directed transmission graph.

To achieve the objectives above, we build an ordinary differential equation-recurrent neural network (ODE-RNN)-based continuous-time temporal point process (TPP) to learn the conditional intensity of each alarm type at any timestamp given historical alarms. The intensities predicted by neural TPP are further utilized to classify whether an alarm is a root alarm by incorporating the prior intensity of each alarm type. To identify the direct causative alarms, we introduce the attribution method, an explainable AI techniques [11], [12], to measure the Granger causality from each past alarm on the evaluated intensities of new alarms. The experimental section evaluates the NEC using both a synthetic event dataset and a large real-world dataset. The real-world dataset consists of 125384 alarm events, which are exported from the Huawei Shennong Intelligent Maintenance and Operation Center (IMOC), a platform deployed at one of China's largest airports and manages over 200000 entities, but only a few thousand nodes have raised alarms. We also create a synthetic event dataset that simulates and simplifies the alarm events in a network system. Results obtained from the two datasets demonstrate that NEC outperforms most state of the art (SOTA) TPP models in modeling alarm events and surpasses general RCA methods in terms of identifying root alarms and recovering transmission chains of anomalies.

The contributions are summarized as follows.

1) We propose an unsupervised, nonintrusive, and log-event-driven RCA technique named NEC for large network systems, where topologies are unknown and the number of entities is extremely large. The implementation of the low-barrier NEC is feasible as long as historical alarm events with annotated timestamps and event types are available. The source code and a synthetic dataset for reproduction are published.[1]
2) NEC employs a novel continuous-time neural TPP model to fit complicated excitation between distinct alarm events and integrates the evaluated conditional intensities from the TPP and the attribution method to identify root alarms and causative alarms.
3) NEC is applied and verified in Huawei Shennong IMOC, a large network system managing over 200000 entities, such as servers, routers, clusters, and containers.

[1]https://github.com/yuanzhaolin/NetEventCause

II. Related Work

In most private cloud systems, the calling chains and topology are usually incomplete because of nonstandardized management [3]. To recover the missed relations between entities, some studies introduce causal discovery algorithms to identify the missed topology. Previous studies represent the alarm events of each entity as a time series, defined as the number of reported alarms in time bins. Temporal causal discovery, such as PC [5] and Granger causality [6], is further utilized on these multidimensional time series to find the causation relationships between entities. Similarly, the event model is also employed to estimate the conditional intensity of alarm events for each entity and assess the mutual excitation between entities [7], which serves as an approximation of the missed topology. The causal graphs generated by these methods consist of vertices that represent entities, such as services or devices, and the directed edges that model the propagation of anomalies. The candidate entities that may potentially induce the root causes are identified by traversing the causality graphs using random walk algorithms [13], [14], [15].

However, existing methods face challenges in discovering the event causality in large private cloud network systems. First, most causal discovery algorithms operate on entities and rely on sufficiently frequent triggered alarms to generate accurate causality graph between entities [5]. However, the number of triggered alarms on the abnormal entities is usually long tail and sparse, so that inadequate training data will degrade the accuracy of the restored topology. Additionally, existing techniques cannot handle the cold start scenarios [16]. When a new entity raises its first alarm, it is challenging to analyze the root cause of this alarm due to the lack of mined causal relations between this node and other entities. Furthermore, a large network system may contain tens of thousands of entities, making the dimension of time series and event types too high for most causal discovery algorithms to handle [17]. These challenges motivate us to develop an RCA technique that should meet two conditions: 1) applicability in large network systems without topology and 2) generalizability to new entities. By comparing the Granger causal test on time series of error counts in time bins and graph event models [18] in modeling log events, previous studies demonstrate that the event model yields the best causal graphs [7]. This finding inspires us to follow the pipeline of modeling anomalies and analyzing root causes based on the original alarm event sequences. Many existing event models make idealized assumptions about the variation of event intensities [19], [20], [21], [22]. Hawkes process [23] assumes that the excitation between two events gradually decays with the increase of their

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXAMPLE OF FOUR TRIGGERED ALARMS

Alarm type (at)	Alarm description(ad)	Entity type(et)	Timestamp(t)
Abnormal communication between management server and elements	[Alarm]WJ-AF-XQ2-FQ37-JR550-KZX17_01; ip=xx.xx, protocol=SNMP,port=161; abnormal communication	Router	2021-01-12 09:48:38
Abnormal communication between management server and elements	[Alarm]WJ-AF-XQ2-FQ37-JR550-KZX17_01; ip=xx.xx, protocol=SNMP,port=161; abnormal communication	Router	2021-01-12 09:48:38
Average memory utilization exceeds the limit	[Alarm] IT-ADC-FQ-2288-S3-16-20U; Average memory utilization; Average memory utilization exceeds the limit	Physical Server	2021-01-12 09:50:01
Link disconnected	[Critical]WJ-AF-XQ2-HJ5720_01; Interface=Eth-Trunk22,index=188; Link disconnected"	Switch	2021-01-12 09:50:22

time difference. Proximal graphical event model [7] assumes constant conditional intensities in the influential time windows. As a neural TPP, SPNPP [10] introduces mixed Gaussian bases to predict intensities. However, the time difference between a root and its derivative alarm is influenced by various factors, such as the self-check period and network delay. This leads that the intensity of the actual alarm event in the network system does not adhere to the assumptions made by most existing event models. As a key technical contribution, NEC models the multivariate TPPs of alarm events using a continuous-time neural network [24], [25], [26] without making any assumptions about event intensities.

III. PROBLEM DEFINITION

This section gives the formulation of RCA from the perspective of events. The Nomenclature lists the key notations used throughout this article. Taking the studied IMOC dataset as an example, we reserve four key fields for each alarm event: alarm type, alarm description, entity type, and timestamp for each triggered alarm. Table I illustrates several samples of alarm logs.

A streaming alarm sequence s consists of $|s|$ alarm events sorted by occurring timestamp, denoted as $\{\mathbf{e}_i = (t_i, k_i)\}_{i=1}^{|s|}$. For each event, $t_i \in \mathbb{R}_+$ is the normalized timestamp of the ith event, and $k_i \in \mathcal{M}$ is the corresponding event type. To achieve two objectives: 1) distinguishing different alarms and 2) generalizing the learned TPP on new entities with the same entity type, we define the event type of an alarm $\mathbf{a}_i$ as the combination of its *alarm type* and *entity type*, viz., the tuple ($\mathbf{a}_i.at$, $\mathbf{a}_i.et$). It is carefully considered to combine the alarm type and entity type as the event type. An alarm occurring in one entity is sometimes caused by a root alarm from other entities. Since the topology of network entities is unknown and many alarms are triggered from new devices, the identity of entities, such as entity name and ID, cannot provide meaningful information for RCA. The only strategy is to omit the identity and utilize its *entity type, alarm type, and triggering times* to analyze the propagation chain. According to our hypothesis, the occurrences of alarms with the same *alarm type* and *entity type* usually follow a similar mechanism, even if they originate from different entities. This assumption is also a prerequisite for the effectiveness of NEC.

Each alarm $\mathbf{e}_i$ is either a root alarm or a derivative alarm of other past causative alarms. We denote the past causative alarm set of $\mathbf{e}_i$ as $\mathbf{R}^{\mathbf{e}_i}$. It is assumed that the recorded triggering timestamps are accurate, and causative alarms always occur earlier than their derivative alarms, viz., $\forall \mathbf{e}_j \in \mathbf{R}^{\mathbf{e}_i}$ s.t. $t_j < t_i$. Anomaly propagation can occur at multiple levels, meaning that a causative alarm may have its own causative alarm set. This study aims to recover the directed anomaly propagation among alarms, which is represented as a directed acyclic graph. Specifically, the analysis algorithm focuses on two objectives: 1) distinguishing root alarms and derivative alarms and 2) identifying the possible causative alarm set $\mathbf{R}^{\mathbf{e}_i}$ for each alarm $\mathbf{e}_i$.

IV. NETEVENTCAUSE

The workflow of NEC is illustrated in Fig. 1. The training and inference of NEC consists of the following steps.

1) We first train a neural continuous-time TPP [27] by offline alarm dataset to model the multivariate alarm event sequences in an unsupervised manner.
2) Next, by utilizing the conditional intensity predicted by neural TPP given historical alarms, we estimate the conditional probability of a new alarm occurring as a root alarm.
3) Finally, we leverage the differentiability of neural TPP to measure the contribution of past events to subsequent alarms using the attribution method, to discover the Granger causalities between alarms.

A. Modeling Alarm Events as a Multivariate TPP

We start by introducing NEC from modeling the alarm event sequences by a continuous-time neural TPP based on the encoder-decoder architecture. The purpose of building this model is to predict the event intensity $\lambda_k(t|\mathcal{H}(t))$ at any time t given the historical triggered alarm events $\mathcal{H}(t)$. The model is trained entirely in an unsupervised manner, obviating the need for annotated datasets from IT experts.

In the encoding stage, we first transform each event with type k_i into a feature vector $\mathbf{v}_{k_i} = \mathbf{V}[k_i]$. The parameter dictionary $\mathbf{V}$ stores changeable keys and parameters, which is useful for incremental learning. Next, the hidden state $\mathbf{h}(t)$

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

defined in continuous-time domain is inferred by an ODE-RNN [24]

$$\begin{cases} \mathbf{h}\left(t_i^-\right) = \text{ODESolve}\ \left(\mathbf{h}\left(t_{i-1}^+\right), \mathbf{f}_\theta, t_{i-1}, t_i\right) \\ \qquad\qquad = \mathbf{h}\left(t_{i-1}^+\right) + \displaystyle\int_{t_{i-1}}^{t_i} \mathbf{f}_{\theta_1}\left(\mathbf{h}\left(t\right)\right) \mathrm{d}t \\ \mathbf{h}\left(t_i^+\right) = \text{RNN}_{\theta_2}\left(\mathbf{h}\left(t_i^-\right), \mathbf{v}_{k_i}\right) \end{cases} \tag{1}$$

where the derivative net $\mathbf{f}_\theta$ and the RNN_{θ_2} are trainable modules. It is important to note that $\mathbf{h}(t_i^-)$ represents the hidden state at t_i before the event feature at t_i is fed into the model. The hidden state $\mathbf{h}(t_i^+)$ represents the updated embedding at t_i after the TPP becomes aware of an event with type k_i occurred at time t_i.

In the decoding stage, the hidden state $\mathbf{h}(t_i^-)$ is fed to a neural network for predicting an intermediate query embedding $\mathbf{q}(t_i)$. At arbitrary timestamp t_i, the logarithmic conditional intensity of any event type k is estimated as the dot product between the embedding of event-type features $\psi(\mathbf{v}_k)$ and $\mathbf{q}(t_i)$

$$\log \lambda_k\left(t_i\right) = \mathbf{q}\left(t_i\right) \cdot \psi\left(\mathbf{v}_k\right), \quad \mathbf{q}\left(t_i\right) = \text{NN}_{\theta_3}\left(\mathbf{h}\left(t_i^-\right)\right). \tag{2}$$

By recursively feeding the features of historical event $\mathcal{H}(t)$ into the ODE-RNN, the trained model can predict the conditional intensity of event type k at timestamp t, denoted as $\lambda_k(t|\mathcal{H}(t))$.

The neural encoder-decoder model is trained by minimizing the expectation of negative log likelihood (NLL) over all sequences in the dataset $\mathcal{S}$

$$\mathcal{L}_{\text{NLL}}\left(\Theta\right) = \mathop{\mathbb{E}}_{s\in\mathcal{S}} \sum_{i=1}^{|s|} \left[-\log \lambda_{k_i^s}^s\left(t_i^s\right) + \sum_{k\in\mathcal{M}} \int_{t_{i-1}^s}^{t_i^s} \lambda_k^s\left(t'\right) \mathrm{d}t' \right] \tag{3}$$

where Θ denotes the trainable parameters, viz., $\{\theta_1, \theta_2, \theta_3, \psi, \mathbf{V}\}$.

We go back to Section III where we define the event type as the combination of the *alarm type* and *entity type*. Intuitively, our definition of event type is improper when two different entities belonging to the same *entity type* simultaneously report alarms with the same *alarm type*. In this case, two alarm events cannot be distinguished by their event types under our definition. Due to the learnable statistical characteristics of the time difference between the causative alarms and the derivative ones with specific entity types in a network system, not only does our neural TPP learn the excitation between event types, but also the influence of time difference on conditional intensity is learned. When entities with the same *et* raise alarms with the same *at*, NEC has awareness in the time domain to distinguish these alarms.

B. Identifying Root Alarms by Comparing Conditional Intensity and Prior Intensity

To differentiate between root alarms and derivative alarms from streaming alarms, we introduce a latent discrete random variable $c \in \{0, 1\}$. This variable indicates whether $\mathbf{e}_i$ is a root alarm ($c = 0$) or a derivative alarm ($c = 1$). Given the historical events $\mathcal{H}(t_i)$ and the occurrence of event $\mathbf{e}_i$ at time t_i as conditions, inferring c is equivalent to estimating a conditional probability

$$P\left(c = 0|\mathbf{e}_i, \mathcal{H}\left(t_i\right)\right) = \frac{P\left(c = 0, \mathbf{e}_i|\mathcal{H}\left(t_i\right)\right)}{P\left(\mathbf{e}_i|\mathcal{H}\left(t_i\right)\right)}$$

$$= \frac{P\left(c = 0, \mathbf{e}_i\right)}{P\left(c = 0, \mathbf{e}_i\right) + P\left(c = 1, \mathbf{e}_i|\mathcal{H}\left(t_i\right)\right)}. \tag{4}$$

We use $P(c, \mathbf{e}_i|\text{condition})$ to represent the possibility that the event $\mathbf{e}_i$ identified by c occurs precisely at time t_i given the condition. When $c = 0$, the condition $\mathcal{H}(t_i)$ is omitted because the probability of a root event occurring is irrelevant to historical events.

The probability that an event occurs at a specific time t_i can be represented as the probability that the number of event occurrences in an infinitesimal time interval $[t_i, t_i + \Delta]$ is greater than zero. This probability is modeled by the Poisson distribution with parameter λ. Therefore, we have

$$P\left(N\left([t_i, t_i + \Delta t]\right) > 0\right) = 1 - e^{-\lambda \Delta t} \tag{5}$$

where λ is the event intensity at time t_i. As the limit theorem $\lim_{x\to 0}[1 - e^{-x}] = x$ holds, we rewrite (4) as

$$P\left(c = 0|\mathbf{e}_i, \mathcal{H}\left(t_i\right)\right) = \frac{\bar{\lambda}_{k_i}}{\bar{\lambda}_{k_i} + \lambda_c} \tag{6}$$

where $\bar{\lambda}_{k_i}$ is a parameter in $\bar{\mathbf{\Lambda}} = [\bar{\lambda}_1, \ldots, \bar{\lambda}_{|\mathcal{M}|}]^T$, representing the intensity that an event with event type k_i occurring as an individual root event and $\lambda_c = \lambda_{k_i}(t_i|\mathcal{H}(t_i)) - \bar{\lambda}_{k_i}$ representing the conditional intensity contributed by historical events. The new introduced parameter $\bar{\mathbf{\Lambda}}$ can be estimated by

$$\bar{\lambda}_k = \arg\max_{\lambda} \left[\mathcal{N}_k * \log \lambda - T * \lambda\right] = \frac{\mathcal{N}_k}{T} \tag{7}$$

where T represents the total duration of the training dataset, and $\mathcal{N}_k$ represents the total count of events of type k occurring as root events. To estimate $\mathcal{N}_k$, professional IT operators need to annotate the root events from a small alarm dataset and estimate the ratio of root events for each event type. The $\mathcal{N}_k$ can be estimated by multiplying the ratio of root events in the small dataset by the total count of events of type k appearing in the entire training dataset.

We consider the alarm $\mathbf{e}_i$ as a root alarm if the probability given by (6) exceeds a given threshold δ. The threshold δ for determining root alarms is defined as 0.2 to ensure a high recall ratio. Obviously, the combination of δ and $\bar{\lambda}_k$ determines the equilibrium between recall ratio and accuracy. From an engineering standpoint, it is allowable to modify the prior intensity $\bar{\lambda}_k$ and the threshold δ to achieve a desired recall ratio, obviating the need for manual annotation of the root events, although it will lead the lost of rigorous definition of $P(c = 0|\mathbf{e}_i, \mathcal{H}(t_i))$.

C. Discovering the Causal Relationships by Attribution Method

In this section, inspired by the previous event Granger causality discovery algorithm [10], we introduce the attribution method [28], an explainable AI technique to discover the causal relationships among events. For any function, the attribution method can evaluate the individual contribution of each input attribute to the output scalar value. Because the conditional intensity has been modeled by the proposed neural TPP, it is intuitive to employ the attribution method to identify the past events in $\mathcal{H}(t)$ that significantly contribute to the

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

occurrences of events at time t. The most contributing past alarms are identified as the causative alarms.

Note that we have trained a TPP that can encode the previous events $\mathcal{H}(t_i)$ and predict the conditional event intensity $\lambda_k(t_i|\mathcal{H}(t_i))$ at timestamp t_i. Attribution method takes in a function $f : \mathbb{R}^m \mapsto \mathbb{R}$, an input $\mathbf{x} \in \mathbb{R}^m$, and a baseline $\underline{\mathbf{x}} \in \mathbb{R}^m$, and measures the contribution of each feature $\mathbf{x}_{1\le i\le m}$ to prediction $f(\mathbf{x})$ relative to $f(\underline{\mathbf{x}})$.

When we analyze the causes of the event $\mathbf{e}_i$, the conditional intensity of event $\mathbf{e}_i$ predicted by neural TPP is regarded as the target function f. The input $\mathbf{x}$ is defined as the concatenation of the feature embedding and timestamps of historical events $\mathbf{e}_1 \cdots \mathbf{e}_i$, viz.,

$$\mathbf{x} \triangleq \left[\mathcal{H}(t_i), t_i, \mathbf{v}_{k_i}\right] \triangleq \left[t_1, \mathbf{v}_{k_1}, \ldots, t_{i-1}, \mathbf{v}_{k_{i-1}}, t_i, \mathbf{v}_{k_i}\right]. \tag{8}$$

The baseline $\underline{\mathbf{x}}$ is defined to have the same size as $\mathbf{x}$, with the event embedding substituted as zero vectors

$$\underline{\mathbf{x}} \triangleq \left[\underline{\mathcal{H}}(t_i), t_i, \mathbf{v}_{k_i}\right] \triangleq \left[t_1, \mathbf{0}, \ldots, t_{i-1}, \mathbf{0}, t_i, \mathbf{v}_{k_i}\right]. \tag{9}$$

Thanks to the differentiability of the learned neural TPP, we choose integrated gradients (IGs) [29], a classical gradient-based attribution method, to generate the contribution vector $\mathbf{A}_i$, which represents the excitation from $\mathcal{H}(t_i)$ to the occurrence of event $\mathbf{e}_i$

$$\begin{aligned}\mathbf{A}_i &= \mathrm{IG}\left(f, \mathbf{x}, \underline{\mathbf{x}}\right) \\ &\triangleq \left(\mathbf{x} - \underline{\mathbf{x}}\right) \odot \int_0^1 \left.\frac{\partial f(\tilde{\mathbf{x}})}{\partial \tilde{\mathbf{x}}}\right|_{\tilde{\mathbf{x}}=\underline{\mathbf{x}}+\alpha(\mathbf{x}-\underline{\mathbf{x}})} d\alpha\end{aligned} \tag{10}$$

where $\odot$ is the Hadamard product and $f(\tilde{\mathbf{x}}) = \log \lambda_{k_i}(t_i \mid \tilde{\mathcal{H}})$ is solved by (2) given $\tilde{\mathcal{H}} = \underline{\mathcal{H}}(t_i) + \alpha(\mathcal{H}(t_i) - \underline{\mathcal{H}}(t_i))$. We consider the scalar value $\mathbf{A}_i(j)$ $(j < i)$, which represents the contribution from the event embedding $\mathbf{v}_{k_j}$ in $\mathbf{x}$, as the score for identifying $\mathbf{e}_j$ as a causative event of the alarm event $\mathbf{e}_i$. Finally, NEC identifies the top-K causative alarms and the rth $(1 \le r \le K)$ possible causative alarm is determined as

$$\hat{\mathbf{R}}^{\mathbf{e}_i}[r] = \mathbf{e}_{j*}, \text{ s.t. } \left[\sum_{j=1}^{i-1} \mathbb{I}\left[\mathbf{A}_i(j*) \le \mathbf{A}_i(j)\right]\right] = r. \tag{11}$$

The identified top-K causative alarm sets of two alarms may interact with each other. The intersecting ones are the common roots of the two derivative alarms. If the actual cause of an alarm is a combination of multiple alarms, all of them will appear in the top-K set and receive higher contribution scores.

Because the chosen baseline $\underline{\mathbf{x}}$ has never appeared in the training dataset, the predicted $\lambda_{k_i}(t_i \mid \tilde{\mathcal{H}})$ may be meaningless. Therefore, we introduce regularization to ensure that the predicted intensities, given null events at $t_1 \cdots t_{i-1}$, are close to the prior intensities of each event type, as mentioned in Section IV-B

$$\mathcal{L}_{\mathrm{reg}}(\Theta) = \sum_{s=1}^{S} \sum_{i=1}^{|s|} \sum_{k\in\mathcal{M}} \left|\left|\lambda_k\left(t_i \mid \underline{\mathcal{H}}(t_i)\right) - \bar{\lambda}_k\right|\right|^2. \tag{12}$$

Along with (3), the integrated optimized loss function is defined as follows:

$$\mathcal{L}_{\mathrm{NEC}}(\Theta) = \mathcal{L}_{\mathrm{NLL}}(\Theta) + \mathcal{L}_{\mathrm{reg}}(\Theta). \tag{13}$$

Algorithm 1 Training of Neural TPP in NEC

Input: Event sequences dataset $\mathcal{S}$, initialized parameters Θ and intensity of root events $\bar{\mathbf{\Lambda}}$.
Output: Trained parameters Θ.

```
for each events sequence s ∈ S do
  for each event e_i ∈ s with past events H(t_i) do
    Generate a "null" event sequence H̲(t_i) by replacing the event embeddings in H(t_i) with 0 (Eq. (9)).
    for each event type k ∈ M do
      Calculate λ_k(t_i|H(t_i)) and λ_k(t_i | H̲(t_i)) (Eq. (2)).
    end for
  end for
  Compute the NLL, L_NLL(Θ) (Eq. (3)), and the regularization term L_reg(Θ) (Eq. (12)).
  Update parameters Θ to minimize the total loss L_NEC(Θ).
end for
return Θ.
```

Algorithm 2 Identifying Root Event and Analyzing Causal Events

Input: Event $\mathbf{e}_i$, past events sequence $\mathcal{H}(t_i)$, neural TPP with trained parameters Θ, root event threshold δ (default 0.2), intensity of root events $\bar{\mathbf{\Lambda}}$, and size of the candidate set K.
Output: Type of $\mathbf{e}_i$, list of causal events

```
Infer the conditional intensity λ_{k_i}(t_i|H(t_i)) (Eq. (2)).
Estimate the probability P(c = 0|e_i, H(t_i)) that e_i is a root event, given λ̄_{k_i} (Eqn. (6)).
if P(c = 0|e_i, H(t_i)) ≥ δ then
  return type: Root, []
else
  Compute the contribution vector A_i ∈ R^{|H(t_i)|} (Eq. (10))
  Identify the causal events R̂^{e_i} from H(t_i) that have the top-K contributions in A_i.
  return type: Derivative, [R̂^{e_i}[1], ..., R̂^{e_i}[K]]
end if
```

D. Summary of NEC

Algorithms 1 and 2 provide the pseudocode for the training and utilizing the trained neural TPP to identify root events and causal events, respectively. The time complexity of two algorithms is analyzed as follows. Without loss of generality, we assume the length of each event sequence as N and the times invoking the differential network $\mathbf{f}_{\theta_1}$ in solving the numerical approximation of integration from t_{i-1} to t_i as K. For instance, $K = 4$ is used in the Runge-Kutta method, whereas K is adaptive in our experiments where the Dormand-Prince solver is employed. It is noted that K is considered a constant variable in our analyses. For each event sequence, solving the NLL in Algorithm 1 requires $\mathcal{O}(NK|\mathcal{M}|)$ invocations of $\lambda(t)$, which dominate the overall time consumption. It only requires a single feed-forward pass on the ODE-RNN to solve for all hidden states and conditional intensities at each time step, resulting in the operation cost of $\mathcal{O}(1)$ for calling $\lambda(t)$. In the stage of identifying root events and causes, it costs $\mathcal{O}(1)$ to determine whether an event is a root based on (6).

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

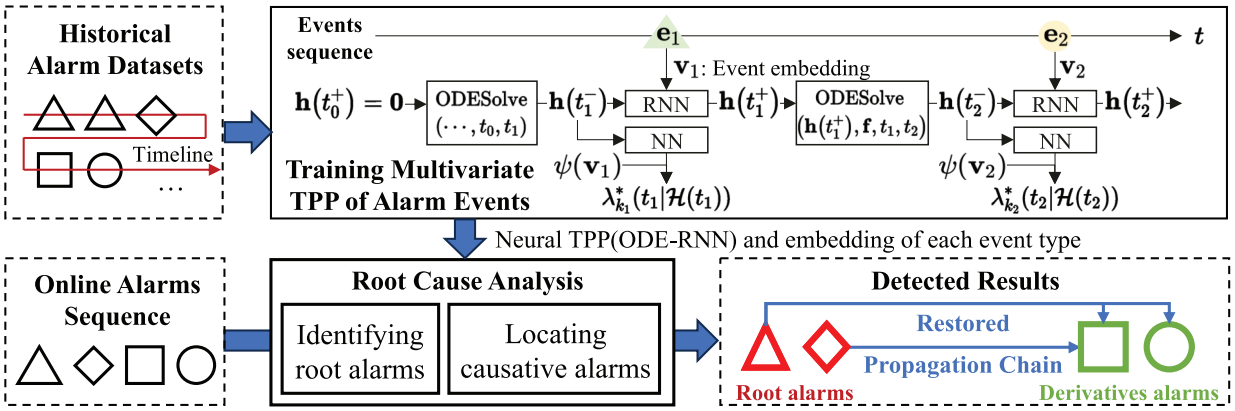


Fig. 1. Overall structure of NEC.

Regarding locating causal events of derivative events, the dominant cost arises from (10), where the contribution vector is individually computed for each event. The computation cost of calculating partial derivative $(\partial f/\partial \tilde{\mathbf{x}})$ is equal to a feed-forward pass over the entire event sequence, incorporating both (1) and (2), and is denoted as $\mathcal{O}(NK)$. Assuming the number of steps for solving the integration in (10) as K', the total time complexity of identifying the causal event set from past events $\mathcal{H}(t_i)$ is $\mathcal{O}(NKK')$. Consequently, the complexity of RCA for the entire event sequence is $\mathcal{O}(N^2KK')$.

Due to the short triggering separation between causal events and derivative events in network systems, there are usually not many events located between two relational events. Therefore, the length N is defined based on the maximum possible transmission time from the root to the derivative. In our real-world application, N is very small (less than 20) and the speed of RCA for each event is significantly faster than the generation of new alarms.

V. Experiments

This section conducts separate evaluations of NEC using both a synthetic event dataset and a large real-world dataset. Two research questions will be answered.

1) Does the proposed continuous-time neural TPP effectively fit the alarm event sequences?
2) How accurate is the NEC in identifying root alarms and locating the causative alarms of derivative alarms?

A. Introduction to the Datasets

1) IMOC Dataset: The real-world dataset is exported from the Huawei Shennong IMOC system from January 2021 to May 2023. The IMOC is deployed in one of China's biggest airports and it manages about 200 000 entities, including network devices, containers, and microservices. The preprocessing of the IMOC dataset consists of two steps: alarm filtering and alarm compression. We initially removed alarm types with low frequencies, those with fewer than five events. Next, the repeated alarms triggered by an identical entity simultaneously are compressed by a designed filtering strategy. Finally, the reserved dataset includes 125 384 alarm events reported from 3634 entities. Fig. 2 illustrates the log-scale frequency distribution of the number of alarms that one entity ever reported. The number of alarms reported by each entity follows a long-tail distribution, with the majority of entities reporting only a few alarms. Among all entities, approximately one-third of them reported only one alarm. By considering the combination of *entity type* and *alarm type* as the event type for learning TPP, all alarms are categorized into 518 event types, which is significantly smaller than the count of entities.

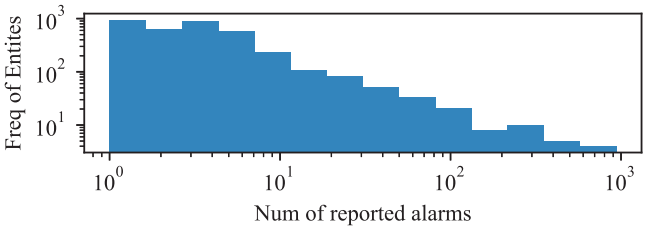


Fig. 2. Frequency distribution histogram of the number of alarms that one entity ever reported in the dataset.

In the training stage of the TPP, we randomly select 10 000 event sequences from the alarm database spanning from January 2021 to December 2022. These sequences are used to train and evaluate the neural TPP in Experiment 1. To construct an event sequence that can be fed into the neural TPP, we sample a starting alarm, denoted as $\mathbf{s}$, from all the sorted alarm events. Subsequently, we search for the farthest alarm event, denoted as $\mathbf{e}$, satisfying that the time difference between $\mathbf{s}$ and $\mathbf{e}$ is less than 24 h. The timestamps and event types of events that occurred between s and e are utilized to construct each event tuple $\mathbf{e}_i = (t_i, k_i)$. Finally, 60% of the 10 000 sequences are reserved for model training, 20% are used for model evaluation, and the remaining 20% is the test dataset.

In the stage on evaluating the accuracy of RCA, we use the event log from January 2023 to May 2023 as the detected alarm sequences, which are not visible during the training process. The ground truths of true causative alarms for each alarm $\mathbf{e}_i$, denoted as $\mathbf{R}^{\mathbf{e}_i}$, are manually annotated by domain experts. Due to the streaming nature of triggered alarms, the event model learned by NEC cannot be directly applied to extremely

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

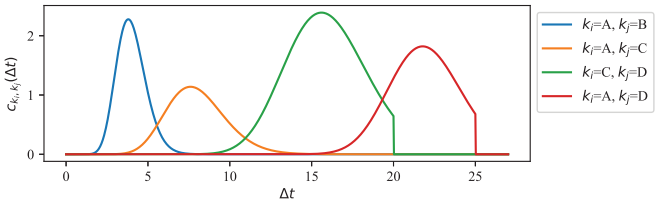


Fig. 3. Event intensity of each event type varies with the time difference Δt. The symbols in the parenthesis represent the causative event types.

TABLE II
DERIVATIVE RELATIONSHIP BETWEEN EVENT TYPES IN THE SYNTHETIC DATASET

Event Type	B	Causative Event Type	α	β	r	W
A	0.5	N/A	N/A	N/A	N/A	N/A
B	0.2	A	20	0.2	5	10
C	0.2	A	20	0.4	5	20
D	0.2	C	40	0.4	15	20
		A	100	0.22	30	25
E	0.3	N/A	N/A	N/A	N/A	N/A

long event sequences due to memory limitations. Therefore, we introduce a sliding window approach to detect root alarms and locate causative alarms in a cascading sequence of alarms. A sliding window of size W is applied to the original sequence of alarms, with a moving distance of d ($d \ll W$). At each step of the sliding window, the alarms within the window are inputted into the detection module, and the detected results from the last range of d are retained.

2) Synthetic Dataset: To evaluate the performance of NEC on a dataset that can be made public, we create a synthetic event dataset that simulates alarm events in a network system. The synthetic event dataset consists of five different event types, marked as A-E. The event types A and E are root event types with intensities determined by the constant values specified by parameter B. The other three event types B-D may occur as root events or derivative events resulting from previous root events. The excitation from a causative event type k_i to a derivative event type k_j is modeled using a scaled probability density function of the gamma distribution

$$c_{k_i,k_j}(\Delta t) = \begin{cases} r * \dfrac{\beta^{\alpha}\Delta t^{\alpha-1}e^{-\beta\Delta t}}{\Gamma(\alpha)}, & t <= W \\ 0, & t > W \end{cases} \tag{14}$$

where Δt denotes the time elapsed since the occurrence of the causative event. The derivative relationships among each event type and the parameters of each conditional intensity function are illustrated in Table II. We also illustrate the curve of event intensities as they change with the time difference Δt in Fig. 3. The total intensity of each event type k at time t is $\lambda_k(t)$

$$\lambda_k(t) = \sum_{\mathbf{e}_i \in \mathcal{H}(t)} c_{k_i,k}(t - t_i) + B. \tag{15}$$

The experimental task is to detect the root events, which occur due to their base intensity, and to identify the sole causative event for each derivative event. Specifically, both A and C are causative event types of event type D. It is challenging to identify the correct causative of D when many events with types A, C, and D are mixed.

Given the functions of event intensities, we employ Ogata's thinning algorithm [30] to simulate the multivariate point process sequences. A total of 1000 sequences are generated, with each sequence having an average length of approximately 100. The sequences are divided into three parts: 700 for training, 100 for validation, and 200 for testing.

Although the synthetic dataset is not as large as the IMOC in the number of event types, it effectively imitates three primary challenges of event-level causality analysis.

1) The conditional intensity of each event type given history is not simply decaying with time, as demonstrated in Fig. 3.
2) Events with the same event type may occur simultaneously, but they were reported from different entities and have different root alarms.
3) A specific event type may occur as either a root event or a derivative event.

We, therefore, suggest experiments on the synthetic dataset are reasonable and sufficient to support the effectiveness of NEC in tacking the challenges in IMOC.

B. Experiment 1: Evaluating Neural TPP

1) Baselines: We compare NEC to the following baselines in fitting alarm events.

1) *SPNPP* [10] and *recurrent point process network (RPPN)* [31] are two state-of-the-art TPP models implemented with neural network. The implementation details and hyperparameters adhere to the settings in the original articles.
2) *HEXP, HSG, and NHPC [9]:* Hawkes process with exponential kernels, Gaussian kernels, and nonparametric Hawkes process. These three statistics-based methods serve as common baselines in temporal event modeling.

2) Experimental Setting: The ODE functions defined with the derivative net $\mathbf{f}$ in (1) and the NLL in (3) are solved by the Dopri5 solver in torchdiffeq package [32]. It is a numerical differential equation solver based on the PyTorch framework. In terms of the hyperparameter setting, the hidden size of trained event feature $\mathbf{v}_k$ for each event type k is 64. The derivative net $\mathbf{f}$ is defined as a single-layer multilayer perceptron (MLP) whose hidden size is 32. In the decoder model, both embedding functions ψ and NN are also single-layer MLPs, whose hidden size and output size are all 64. All activation functions in every neural network are tanh(·). The model is trained by minimizing the expected NLL and the regularization terms over sequences in dataset. The maximal training epoch is 500 and the training process will also be early stopped if the NLL on validation does not decrease for 50 epochs. Adam is chosen as the optimizer whose learning rate is $1e^{-3}$. It took approximately 3 h to train the TPP on a GPU station equipped with an Intel Xeon Silver 4310 processor and a GeForce RTX 3090 with 16-GB memory.

3) Evaluation Metrics: We select two metrics to evaluate the event models in fitting alarm datasets. The first metric,

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

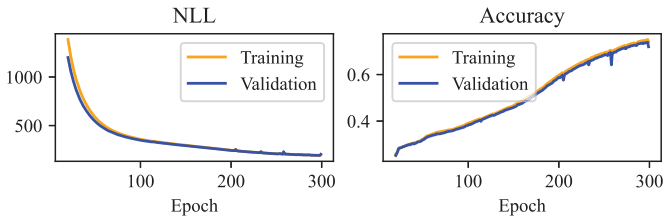


Fig. 4. Training and evaluation losses over epochs.

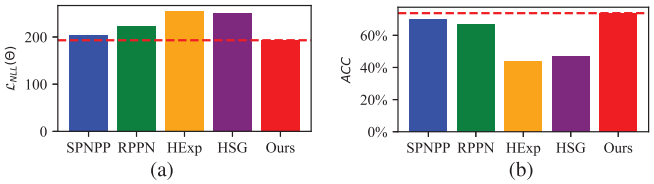


Fig. 5. NLLs and accuracies on a test dataset of various trained event models. (a) NLL. (b) Accuracy.

NLL, assesses the accuracy of predicted conditional intensity on the test dataset. Meanwhile, we use the model to predict the category of the event that occurred at t_i given the historical events sequence before t_i. *Accuracy of the predicted event type* is the second metric for evaluation.

4) Results: Fig. 4 illustrates the NLL losses and the classification accuracy on the training dataset and the validation dataset during the training process. As the curves of the training dataset and validation dataset are close to each other, it demonstrates that the proposed event model fits the temporal event intensities indeed. Fig. 5 shows the comparison of the proposed TPP model and baselines by visualizing the two evaluation metrics on the test dataset of IMOC. We find the model's performance is robust under different random seeds for model initialization. Our neural continuous-time TPP model outperforms the baselines across both metrics. This outcome verifies that the unrestricted ODE-RNN is better suited for fitting complicated time-varying intensities, particularly under the excitation among multiple alarm event types in network systems. A lower NLL also indicates that our model accurately predicts higher intensity for the correct event type at the time points when the alarms occur. This is a critical prerequisite for analyzing the causality between events accurately.

C. Experiment 2: Evaluation of Identifying Root Alarms and Locating Derivative Alarms

The second experiment investigates the effectiveness of NEC in identifying root alarms from a mixture of streaming alarms and locating the causative alarms of each derivative alarm.

1) Evaluation Metrics: Since finding root alarms is a binary classification problem, we use the *AUC*, the area under the receiver operating characteristic (ROC) curve, to evaluate the performance of identifying root alarms.

In terms of locating causative alarms, the accuracy is measured as the ACC@k metric, which is widely used in the RCA community [5], [13]. For an alarm set A whose causative alarms are manually annotated in advance, ACC@k is defined as follows:

$$\text{ACC@}k = \frac{1}{|A|}\sum_{\mathbf{e}_i \in A} \frac{\sum_{1\le r\le k} \hat{\mathbf{R}}^{\mathbf{e}_i}[r] \in \mathbf{R}^{\mathbf{e}_i}}{\min(k, |\mathbf{R}^{\mathbf{e}_i}|)}. \tag{16}$$

Generally, recall is more important than the accuracy in AIOps. This is because engineers can take relatively low cost to verify if a diagnosis from intelligent algorithms is a false positive sample. In the chosen evaluation metric equation (16), the size of causative alarm set, $|\mathbf{R}^{\mathbf{e}_i}|$, is typically equal to 1, indicating ACC@k represents the average recall when the model makes k guesses.

2) Baselines: Since the real topology is unknown and the number of entities is extremely large, most existing RCA algorithms are disabled in this scenario. Therefore, we construct the following methods as baselines.

a) SPNPP + Attribution (SPNPP + A): As an approximation of NEC, we replace the NEC's neural TPP as the SPNPP, introduced in experiment 1. The method for identifying root alarms and causative alarms is consistent with NEC. The comparison between SPNPP + A and NEC is informative to demonstrate the advantage of the proposed ODE-RNN-based TPP model.

b) Frequency Item Set [33]: As the most common RCA method, the approach discovers the association rules between different alarm types using frequent item-set mining. Since frequency item set (FIS) only identifies the associations between event types rather than specific events, we define the kth nearest alarm with the causative alarm type as the located kth possible causative alarm.

c) Peter Clark: Following the MonitorRank [13], we construct time bins of 5 min and count the number of alarms within each bin. This process allows us to generate a time series of alarm counts for each event type. Next, we introduce the improved PC [5], which is a causal discovery algorithm based on conditional independence tests, for identifying the sparse causality between different event types. Similar to the FIS, the generated causal graph describes the causality between event types and we adopt the identical method to quantify the probability that the previous one contributes to the occurrence of the later one.

d) Cause: In addition to the PC algorithm, we also introduce the CAUSE [10], an event Granger causality discovery algorithm, to generate a causality matrix. When the excitation from event type k_i to k_j exceeds 10% of the accumulative excitation to type k_j, their causative relationship is reserved as a direct edge in the constructed causal graph.

e) Random Guess: Random guess (RG) represents the random selection of causative alarms from past alarms within a specific time window. As a lower bound for the task of locating causative alarms, the accuracy of RG indicates the ratio of correct causative alarms to the total number of candidate alarms within the detected window.

3) Results: In Table III, the column with the head AUC illustrates that NEC shows the best AUC value (0.843), in identifying root alarms from mixed alarms of the IMOC dataset. Results obtained from the synthetic dataset also demonstrate

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF NEC AND BASELINES ON THE IMOC DATASET

Algorithm	Identifying Root Events	Locating Causative Events			
	AUC	ACC@1	ACC@2	ACC@3	ACC@4
NEC	**0.843**	**24.2%**	**37.4%**	**42.6%**	**61.2%**
SPNPP+A	0.821	21.6%	35.0%	40.6%	58.2%
FIS	0.72	19.4%	29.5%	30.6%	32.7%
CAUSE	0.694	18.7%	26.9%	38.3%	57.5%
PC	0.688	16.5%	25.8%	36.4%	55.0%
RG	-	5.1%	9.8%	14.9%	18.4%

TABLE IV
COMPARISON OF NEC AND BASELINES ON THE SYNTHETIC DATASET

Algorithm	Identifying Root Events	Locating Causative Events			
	AUC	ACC@1	ACC@2	ACC@3	ACC@4
NEC	**0.93**	**67.6%**	**86.6%**	**97.3%**	**99.6%**
SPNPP+A	0.91	60.1%	83.9%	94.4%	99.0%
FIS	0.863	56.6%	77.6%	88.0%	91.0%

a performance gap between NEC and the baselines, as shown in Table IV. The AUC values of feature importance curve (FIC), PC, and CAUSE, which generate causal graphs at the event-type level, are noticeably lower than the two NEC-based methods working at the event level. The reason is that the static causal graph only models the causality between event types, instead of the causality between specific events. If two alarms occurred successively where their event types are causally correlated but the alarms themselves are irrelevant, a method based on a static graph of event types will mistakenly identify the former alarm as the causative event of the latter alarm. NEC addresses this problem by directly modeling the conditional intensity at the event level, rather than the event-type level. The AUC value slightly decreases when the ODE-RNN in NEC is replaced with SPNPP. This further confirms the finding from experiment 1, demonstrating that the proposed ODE-RNN-based TPP outperforms the baselines in terms of the goodness-of-fit of alarm events. Therefore, our model can estimate a more accurate conditional probability that an alarm is a root event.

In the evaluation of locating causative alarms, we compare the identified $\hat{\mathbf{R}}^{\mathbf{e}_i}$ with the ground truth $\mathbf{R}^{\mathbf{e}_i}$ annotated by experts. The columns ACC@1–ACC@4 in Tables III and IV illustrate the results. In general, NEC outperforms the baseline methods based on the static event-type graph, as well as the combination of other event models and attribution methods. As a sequence-specific analysis method, not only does NEC utilize the information from event types, but the time differences between multiple alarms are critical features to quantify the contributions between events.

D. Balancing Time Cost and Accuracy in RCA

As analyzed in Section IV-D, the time complexity of NEC for analyzing the causes of an event is $\mathcal{O}(NKK')$. The parameter K' is adjustable, allowing control over the accuracy and time required for solving the integration equation (10).

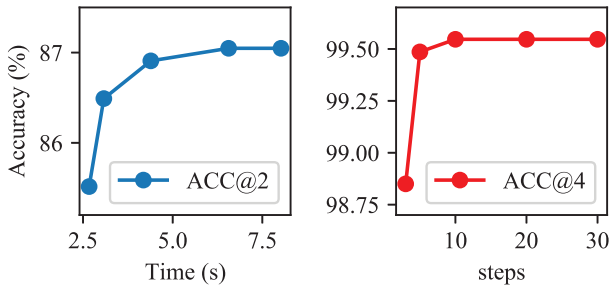


Fig. 6. Relationship between the time consumption and accuracy in RCA.

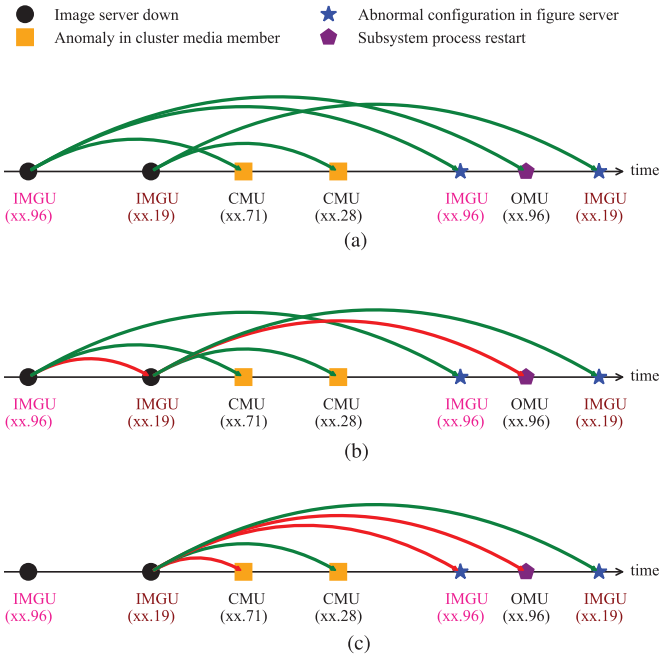


Fig. 7. Visualization of detected causative alarms compared with the ground truth. (a) Ground-truth causative alarms. (b) NEC (ours). (c) CAUSE.

In the ablation study, therefore, we use the synthetic dataset to conduct an ablation study investigating how the number of steps influences the accuracy and time consumption in identifying causal events.

As shown in Fig. 6, the number of steps, K', were varied at 3, 5, 10, 20, and 30, with ACC@2, ACC@4, and time visualized under different step sizes. The results demonstrate that the accuracies are still applicable with fewer steps, even when the K' is reduced to three. The time increases linearly with the number of steps, whereas the improvements in accuracy almost cease when the step size exceeds 20. It concludes that the accuracy of locating causal events is highly robust with respect to both step size and the precision of the integration solution. In practice, reducing the number of steps can achieve faster detection speeds without significantly compromising accuracy.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

E. Case Analysis

Fig. 7 visualizes a representative example of the detected causative alarms and the corresponding ground truth. Each kind of marker with a specific color and geometrical shape corresponds to an event type. In Fig. 7(b) and (c), the directed arcs between alarms represent the discovered causality with the highest attribution score. The arcs that are correctly identified, as compared to the ground-truth association in Fig. 7(a), are marked in green. Conversely, the arcs that incorrectly identify the causality are marked in red. and the wrong ones are marked in red. Alarms that are not connected by any arcs are identified or annotated as root alarms. The entity name of each alarm is displayed under the timeline. As the entities IMGU(xx.96) and IMGU(xx.19) raise multiple alarms, we mark these two names with specific colors. From Fig. 7(a), we observe that two root alarms originating from two image processing units (IMGUs) lead to the occurrences of two derivative alarms within their own entities as well as three other derivative alarms in other entities, including two cluster management units (CMUs) and one operation management unit (OMU).

In Fig. 7(c), CAUSE incorrectly identified the second "black circle" as the cause of the first "blue star" even though these two alarms are not causally related. The reason for this mistake is that methods based on static causal graphs of event types cannot effectively utilize the information from the timestamps when multiple alarms occur. Because the time differences between causative alarms and derivative alarms belonging to specific event types are relatively stable, NEC leverages this property by incorporating the timestamp of each alarm as a critical feature that significantly influences the predicted conditional intensity of the analyzed alarm. When NEC analyzes the causative alarms of the first "blue star," the attribution method not only effectively identifies the "black circle" as the most probable causative alarm type, but also infers the first "black circle" is the most significant contributor in the time domain. NEC can distinguish alarms with the same types originating from different entities and accurately identify the causative alarms of IMGUs and CMUs, where the CAUSE method sometimes fails. As an unsupervised RCA technique, NEC only utilizes the information from triggering timestamp and event type to detect root causes. This ability is crucial for generalizing the technique to other private cloud systems without the need for manual annotations and network topology.

VI. Conclusion

This article proposes the NEC for RCA in large network systems, where the topology among devices and services is unknown. As a nonintrusive and unsupervised log-event-based RCA technique, NEC utilizes an ODE-RNN as a backbone to model the occurrences of temporal alarm events as a multivariate TPP. The trained TPP can evaluate the conditional intensities of each alarm given the alarms occurred previously. The predicted intensities are employed to identify the root alarms by Bayesian inference and local causative alarms by attribution method. We evaluate the performance of NEC using a synthetic dataset and a real alarm event dataset exported from the Huawei Shennong IMOC and compare NEC with existing event models and RCA methods. The experimental results indicate that the proposed ODE-RNN-based neural TPP outperforms the existing SOTA neural TPPs in the goodness-of-fit of modeling alarm events. Meanwhile, NEC shows excellent accuracies in identifying root alarms (AUC =0.843) and finding causative alarms (ACC@3 = 42.6% and ACC@4 = 61.2%).

The accuracy of NEC is currently limited because it only utilizes the information on timestamps and event types. In particular, NEC is ineffective when it encounters new alarm types. The only available solution for the current NEC is to allocate an initial embedding for the new type and subsequently fine-tuning the event embedding through the observation of future event sequences that include this type of event. In future research, we aim to overcome this bottleneck by incorporating additional modalities. For instance, the alarm logs, expressed in text, can be employed to generate event embeddings through the application of language encoders, such as BERT [34], [35]. The time-series metric of each device, such as CPU utilization rate and time delay, are also useful to infer the prior granger casual graph between entities [5].

References

[1] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 1–39, Feb. 2022.

[2] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-world challenges and research innovations," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., Companion Proc. (ICSE-Companion)*, May 2019, pp. 4–5.

[3] P. Wang et al., "CloudRanger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. Clust. Cloud Grid Comput.*, Aug. 2018, pp. 492–502.

[4] Á. Brandón, M. Solé, A. Huélamo, D. Solans, M. S. Pérez, and V. Muntés-Mulero, "Graph-based root cause analysis for service-oriented and microservice architectures," *J. Syst. Softw.*, vol. 159, Jan. 2020, Art. no. 110432.

[5] Y. Meng et al., "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. Quality Serv. (IWQoS)*, Sep. 2020, pp. 1–10, doi: 10.1109/IWQoS49365.2020.9213058.

[6] Y. Liu, H.-S. Chen, H. Wu, Y. Dai, Y. Yao, and Z. Yan, "Simplified Granger causality map for data-driven root cause diagnosis of process disturbances," *J. Process Control*, vol. 95, pp. 45–54, Nov. 2020.

[7] V. Arya, K. Shanmugam, P. Aggarwal, Q. Wang, P. Mohapatra, and S. Nagar, "Evaluation of causal inference techniques for AIOps," in *Proc. 3rd ACM India Joint Int. Conf. Data Sci. Manage. Data (8th ACM IKDD CODS 26th COMAD)*, Jan. 2021, pp. 188–192.

[8] J. Qiao, R. Cai, S. Wu, Y. Xiang, K. Zhang, and Z. Hao, "Structural Hawkes processes for learning causal structure from discrete-time event sequences," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, Aug. 2023, pp. 5702–5710.

[9] A. G. Hawkes, "Spectra of some self-exciting and mutually exciting point processes," *Biometrika*, vol. 58, pp. 83–90, Apr. 1971.

[10] W. Zhang, T. K. Panum, S. Jha, P. Chalasani, and D. Page, "CAUSE: Learning Granger causality from event sequences using attribution methods," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, Jan. 2020, pp. 11235–11245.

[11] P. Linardatos, V. Papastefanopoulos, and S. Kotsiantis, "Explainable AI: A review of machine learning interpretability methods," *Entropy*, vol. 23, no. 1, p. 18, 2020.

[12] C. Chen et al., "BALANCE: Bayesian linear attribution for root cause localization," *Proc. ACM Manage. Data*, vol. 1, no. 1, pp. 1–26, May 2023.

[13] M. Kim, R. Sumbaly, and S. Shah, "Root cause detection in a service-oriented architecture," in *Proc. ACM SIGMETRICS/Int. Conf. Meas. Model. Comput. Syst.*, Jun. 2013, pp. 93–104.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.

[14] D. Wang, Z. Chen, Y. Fu, Y. Liu, and H. Chen, "Incremental causal graph learning for online root cause analysis," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 2269–2278.
[15] M. Li et al., "Causal inference-based root cause analysis for online service systems with intervention recognition," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 3230–3240.
[16] M. Grbovic, W. Li, N. A. Subrahmanya, A. K. Usadi, and S. Vucetic, "Cold start approach for data-driven fault detection," *IEEE Trans. Ind. Informat.*, vol. 9, no. 4, pp. 2264–2273, Nov. 2013.
[17] Z. Hao, H. Zhang, R. Cai, W. Wen, and Z. Li, "Causal discovery on high dimensional data," *Appl. Intell.*, vol. 42, no. 3, pp. 594–607, Apr. 2015.
[18] D. Bhattacharjya, D. Subramanian, and T. Gao, "Proximal graphical event models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, Jan. 2018, pp. 8136–8145.
[19] O. Shchur, A. C. Türkmen, T. Januschowski, and S. Günnemann, "Neural temporal point processes: A review," in *Proc. 30th Int. Joint Conf. Artif. Intell.*, Aug. 2021, pp. 4585–4593.
[20] J. Yu, Y. Lee, K. C. Yow, M. Jeon, and W. Pedrycz, "Abnormal event detection and localization via adversarial event prediction," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 8, pp. 3572–3586, Aug. 2022.
[21] P. Chapfuwa et al., "Calibration and uncertainty in neural time-to-event modeling," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1666–1680, Apr. 2023.
[22] W. Wu, H. Liu, X. Zhang, Y. Liu, and H. Zha, "Modeling event propagation via graph biased temporal point process," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1681–1691, Apr. 2023.
[23] H. Mei and J. Eisner, "The neural Hawkes process: A neurally self-modulating multivariate point process," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Dec. 2017, pp. 6757–6767.
[24] Y. Rubanova, R. T. Q. Chen, and D. K. Duvenaud, "Latent ordinary differential equations for irregularly-sampled time series," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32. Red Hook, NY, USA: Curran Associates, Aug. 2019.
[25] Z. Yuan, Y. Wang, X. Ban, C. Ning, H.-N. Dai, and H. Wang, "Autonomous-Jump-ODENet: Identifying continuous-time jump systems for cooling-system prediction," *IEEE Trans. Ind. Informat.*, vol. 19, no. 7, pp. 7894–7904, Jul. 2022.
[26] Z. Yuan, X. Ban, Z. Zhang, X. Li, and H. Dai, "ODE-RSSM: Learning stochastic recurrent state space model from irregularly sampled data," in *Proc. 37th AAAI Conf. Artif. Intell.*, vol. 37, Jun. 2023, pp. 11060–11068.
[27] P. Dutta, K. Mayilvaghanan, P. Sinha, and A. Dukkipati, "Deep representation learning for prediction of temporal event sets in the continuous time domain," in *Proc. 15th Asian Conf. Mach. Learn.*, vol. 222, B. Yanıkoğlu and W. Buntine, Eds., Nov. 2023, pp. 343–358.
[28] M. Ancona, E. Ceolini, C. Öztireli, and M. Gross, "Gradient-based attribution methods," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Cham, Switzerland: Springer, 2019, pp. 169–191.
[29] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proc. Int. Conf. Mach. Learn.*, vol. 70, 2017, pp. 3319–3328.
[30] Y. Chen, "Thinning algorithms for simulating point processes," Florida State Univ., Tallahassee, FL, USA, Tech. Rep., 2016.
[31] S. Xiao, J. Yan, M. Farajtabar, L. Song, X. Yang, and H. Zha, "Learning time series associated event sequences with recurrent point process networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 10, pp. 3124–3136, Oct. 2019.
[32] R. T. Q. Chen. (2018). *Torchdiffeq*. [Online]. Available: https://github.com/rtqichen/torchdiffeq
[33] F. Lin, K. Muzumdar, N. Pavlovich Laptev, M.-V. Curelea, S. Lee, and S. Sankar, "Fast dimensional analysis for root cause investigation in a large-scale service environment," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 48, no. 1, pp. 25–26, Jul. 2020.
[34] W. Hu, G. Yang, Y. Li, W. Cao, and M. Wu, "Root cause identification of industrial alarm floods using word embedding and few-shot learning," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 1465–1475, Feb. 2023.
[35] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th Eur. Conf. Comput. Syst.*, Apr. 2024, pp. 674–688.



Zhaolin Yuan received the Ph.D. degree in computer science from the University of Science and Technology Beijing, Beijing, China, in 2023.

He is currently an Associate Professor with the Institute of Artificial Intelligence, University of Science and Technology Beijing. He has conducted research projects in the field of artificial intelligence (AI) for network modeling, smart manufacturing and mining with substantial government, and industrial grants. He has published more than ten articles in top AI conferences including AAAI and leading international journals including IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE/CAA JOURNAL OF AUTOMATICA SINICA, *Acta Automatica Sinica*, and *Tunnelling and Underground Space Technology*. He has also obtained dozens of patents and copyrights. His research interests include AI for IT operations (AIOps), intelligent manufacturing, AI in fashion, industrial system optimization, and model-based reinforcement learning.



Long Ma received the Ph.D. degree in computer science from Delft University of Technology (TU Delft), Delft, The Netherlands, in 2022.

He is currently a Research Engineer with Huawei Technologies Company Ltd., Shenzhen, China. His main interests include network modeling and prediction.



Wenjia Wei received the Ph.D. degree in information and communication engineering from the University of Science and Technology of China (USTC), Hefei, China, in 2020.

He is currently a Research Engineer with Huawei Technologies Company Ltd., Shenzhen, China. His main interests include network modeling and simulation.



Xia Zhu received the Ph.D. degree in computer science and engineering from Southeast University, Nanjing, China, in 2014.

He is currently a Technical Expert with Huawei Technologies Company Ltd., Shenzhen, China. His main interests include network modeling and optimization.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.



Mingjie Sun received the bachelor's degree from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2016, the master's degree from Xidian University, Xi'an, China, in 2019, and the Ph.D. degree from the University of Liverpool, Liverpool, U.K., in 2022.

He is currently an Associate Professor with the School of Computer Science and Technology, Soochow University, Suzhou, China. As the project leader, he has undertaken multiple projects funded by the National Natural Science Foundation of China and provincial science and technology funds. His research directions mainly include computer vision, multimodal computing, and reinforcement learning.



Duxin Chen received the B.S. degree in automatic control and the Ph.D. degree in control science and engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2013 and 2018, respectively.

He is currently an Associate Professor with the School of Mathematics and the Vice Director of Jiangsu Provincial Scientific Research Center of Applied Mathematics, Southeast University, Nanjing, China. He is also one of the direction leaders of the Huawei-SEU Joint Innovation Laboratory of Networked Collective Intelligence, Nanjing. His research interests include causal inference, prediction/generation, system identification techniques for complex networks and system sciences, and artificial intelligence-related theory and applications.



Xiaojuan Ban received the Ph.D. degree from the University of Science and Technology Beijing, Beijing, China, in 2003.

She is currently a Ph.D. Supervisor with the University of Science and Technology Beijing. She is also the Managing Director of Chinese Association for Artificial Intelligence, Beijing. She has authored more than 300 articles.

Dr. Ban is a member of the Standing Committee of the human-computer interaction specialty and the theoretical computer science specialty in China Computer Federation.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 23,2026 at 05:38:39 UTC from IEEE Xplore. Restrictions apply.